

Example 7

```
1 public void function(int n){
2   int i, j, k, count = 0;
3   for(i = n/2; i <= n; i++)
4     for(j = 1; j + n/2 <= n; j++)
5       for(k = 1; k <= n; k = k * 2)
6         count++;
7 }
```

Complexity analysis:

- Loop i: chạy từ $n/2$ đến $n \Rightarrow$ khoảng $n/2$ lần $\Rightarrow O(n)$.
- Loop j: điều kiện $j + n/2 \leq n \Rightarrow j \leq n/2$, j bắt đầu từ 1, mỗi lần tăng 1 $\Rightarrow O(n/2) = O(n)$.
- Loop k: k chạy 1, 2, 4, ..., $n \Rightarrow O(\log n)$.
- Tổng độ phức tạp: $O(n \times n \times \log n) = O(n^2 \log n)$.

Example 8

```
1 public void function(int n) {
2   int i, j, k, count = 0;
3   for(i = n/2; i <= n; i++)
4     for(j = 1; j <= n; j = 2 * j)
5       for(k = 1; k <= n; k = k * 2)
6         count++;
7 }
```

Complexity analysis:

- Loop i: $O(n)$.
- Loop j: j nhân đôi mỗi bước $\Rightarrow O(\log n)$.
- Loop k: $O(\log n)$.
- Tổng: $O(n \times \log n \times \log n) = O(n \log^2 n)$.

Example 9

Xác định độ phức tạp của các hàm sau:

- (a) $T(n) = n \log n + 3n + 2 \Rightarrow$ Bậc cao nhất $n \log n \Rightarrow O(n \log n)$.
- (b) $T(n) = n \log(n!) + 5n^2 + 6 \Rightarrow n \log(n!) \approx n(n \log n) = n^2 \log n$ (theo Stirling), bậc cao nhất $n^2 \log n \Rightarrow O(n^2 \log n)$.
- (c) $T(n) = 100n + 10n^2 \Rightarrow$ Bậc cao nhất $n^2 \Rightarrow O(n^2)$.
- (d) $T(n) = 50n \log n + n^3 + 100n \Rightarrow$ Bậc cao nhất $n^3 \Rightarrow O(n^3)$.
- (e) $T(n) = 10n \log n + n(\log n)^2 \Rightarrow$ Bậc cao nhất $n(\log n)^2 \Rightarrow O(n \log^2 n)$.

Example 10

a)

```
1 public static int example1(int[] arr) {
2   int n = arr.length, total = 0;
3   for (int j=0; j < n; j++)
4     total += arr[j];
5   return total;
6 }
```

Độ phức tạp: $O(n)$.

b)

```
1 public static int example2(int[] arr) {
2   int n = arr.length, total = 0;
3   for (int j=0; j < n; j += 2)
4     total += arr[j];
5   return total;
6 }
```

Vòng lặp chạy $n/2$ lần $\Rightarrow O(n)$.

c)

```
1 public static int example3(int[] arr) {
2   int n = arr.length, total = 0;
3   for (int j=0; j < n; j++)
4     for (int k=0; k <= j; k++)
5       total += arr[j];
6   return total;
7 }
```

Tổng số lần lặp: $\sum_{j=0}^{n-1} (j+1) = \frac{n(n+1)}{2} \Rightarrow O(n^2)$.

d)

```
1 public static int example4(int[] arr) {
2   int n = arr.length, prefix = 0, total = 0;
3   for (int j=0; j < n; j++) {
4     prefix += arr[j];
5     total += prefix;
6   }
7   return total;
8 }
```

Mỗi vòng lặp $O(1)$, tổng n lần $\Rightarrow O(n)$.

e)

```
1 public static int example5(int[] first, int[] second) {
2   int n = first.length, count = 0;
3   for (int i=0; i < n; i++) {
4     int total = 0;
5     for (int j=0; j < n; j++)
6       for (int k=0; k <= j; k++)
7         total += first[k];
8     if (second[i] == total) count++;
9   }
10  return count;
11 }
```

- Vòng lặp i: n lần.
- Vòng lặp j: n lần.

- Vòng lặp k: trung bình $O(n)$ lần (chạy từ 0 đến j).
- Mỗi lần lặp trong cùng làm $O(1)$ công việc.
- Tổng: $O(n \times n \times n) = O(n^3)$.

Câu 4

a. Chọn găng tay

Có tất cả:

- 5 đôi màu đỏ $\Rightarrow$ 10 chiếc đỏ.
- 4 đôi màu vàng $\Rightarrow$ 8 chiếc vàng.
- 2 đôi màu xanh lá cây $\Rightarrow$ 4 chiếc xanh.

Tổng số găng tay: $10 + 8 + 4 = 22$ chiếc.

Trường hợp tốt nhất: Chỉ cần chọn 2 chiếc ngay từ đầu đã tạo thành một đôi.

2

Trường hợp xấu nhất: Ta cần xét số lượng găng tay tối đa có thể chọn mà không có đôi nào hoàn chỉnh. Mỗi đôi gồm 2 chiếc giống nhau. Để không có đôi nào, ta chỉ lấy tối đa 1 chiếc từ mỗi đôi. Có tổng số đôi: $5 + 4 + 2 = 11$ đôi. Số găng tay tối đa không có đôi là 11 (mỗi đôi lấy 1 chiếc). Thêm 1 chiếc nữa (chiếc thứ 12) sẽ bắt buộc tạo thành một đôi với một trong 11 chiếc đã lấy.

Vậy trường hợp xấu nhất cần chọn:

12

b. Tất bị mất

Có 5 đôi tất khác nhau $\Rightarrow$ 10 chiếc tất. Mất 2 chiếc tất bất kỳ, mỗi chiếc có xác suất mất như nhau. Số cách chọn 2 chiếc tất bị mất từ 10 chiếc:

$$\binom{10}{2} = 45$$

Kịch bản tốt nhất: Mất 2 chiếc cùng một đôi $\Rightarrow$ còn lại 4 đôi hoàn chỉnh. Số cách chọn 2 chiếc cùng một đôi: chọn 1 trong 5 đôi $\Rightarrow$ 5 cách. Xác suất kịch bản tốt nhất:

$$P_{\text{tốt nhất}} = \frac{5}{45} = \frac{1}{9} \approx 0.1111$$

Kịch bản xấu nhất: Mất 2 chiếc ở 2 đôi khác nhau, mỗi đôi mất 1 chiếc $\Rightarrow$ còn lại 3 đôi hoàn chỉnh và 2 đôi mất mỗi đôi 1 chiếc. Số cách: chọn 2 đôi khác nhau từ 5 đôi $\Rightarrow \binom{5}{2} = 10$, mỗi đôi mất 1 trong 2 chiếc $\Rightarrow 2 \times 2 = 4$ cách cho hai đôi đó. Tổng số cách:

$$10 \times 4 = 40$$

Xác suất kịch bản xấu nhất:

$$P_{\text{xấu nhất}} = \frac{40}{45} = \frac{8}{9} \approx 0.8889$$

Số đôi tất kỳ vọng còn lại: Gọi X là số đôi còn lại. X có thể là 4 (tốt nhất) hoặc 3 (xấu nhất).

$$E[X] = 4 \times \frac{1}{9} + 3 \times \frac{8}{9} = \frac{4 + 24}{9} = \frac{28}{9} \approx 3.1111$$

$$\boxed{E[X] = \frac{28}{9}}$$

Câu 10: Sự tích bàn cờ

a. Gấp đôi số hạt mỗi ô

Số hạt ở ô thứ k (bắt đầu ô 1): 2^{k-1} . Tổng số hạt trên 64 ô:

$$S = 2^0 + 2^1 + \dots + 2^{63} = 2^{64} - 1$$

Mỗi hạt đếm mất 1 giây. Thời gian đếm:

$$T = 2^{64} - 1 \text{ giây}$$

Quy đổi: 1 năm = $365 \times 24 \times 3600 = 31,536,000$ giây.

$$T \approx \frac{2^{64}}{3.1536 \times 10^7} \text{ năm}$$

$$2^{64} \approx 1.84467 \times 10^{19}.$$

$$T \approx \frac{1.84467 \times 10^{19}}{3.1536 \times 10^7} \approx 5.85 \times 10^{11} \text{ năm}$$

$$\boxed{\approx 585 \text{ tỷ năm}}$$

b. Cộng thêm 2 hạt mỗi ô

Ô thứ nhất: 1 hạt. Ô thứ hai: $1 + 2 = 3$ hạt. Ô thứ ba: $3 + 2 = 5$ hạt. Tổng quát: ô thứ k có $1 + 2(k - 1) = 2k - 1$ hạt.

Tổng số hạt trên 64 ô:

$$\begin{aligned} S &= \sum_{k=1}^{64} (2k - 1) = 2 \sum_{k=1}^{64} k - \sum_{k=1}^{64} 1 \\ &= 2 \times \frac{64 \times 65}{2} - 64 = 64 \times 65 - 64 = 64 \times 64 = 4096 \end{aligned}$$

Thời gian đếm:

$$T = 4096 \text{ giây}$$

Đổi ra phút: $\frac{4096}{60} \approx 68.27$ phút ≈ 1 giờ 8 phút 16 giây.

4096 giây (khoảng 1 giờ 8 phút)

Bài tập 1: Kích thước đầu vào và bậc tăng trưởng

Kích thước đầu vào thường là số lượng phần tử hoặc lượng dữ liệu được truyền vào thuật toán (ví dụ: độ dài mảng, số đỉnh của đồ thị, số bit của một số nguyên).

Bậc tăng trưởng mô tả thời gian chạy hoặc bộ nhớ sử dụng thay đổi thế nào khi kích thước đầu vào tăng lên, bỏ qua các hằng số và các số hạng bậc thấp. Bậc tăng trưởng được biểu diễn bằng các ký hiệu tiệm cận.

Các bậc tăng trưởng thường gặp (từ chậm đến nhanh)

$$O(1) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(2^n) \subset O(n!)$$

Ví dụ: Xác định kích thước đầu vào và bậc tăng trưởng

1. Tính tổng các phần tử mảng:

```
int tong = 0;
for (int i = 0; i < n; i++) tong += arr[i];
```

Kích thước đầu vào: n (độ dài mảng). Bậc tăng trưởng: $O(n)$ (tuyến tính).

2. Tìm kiếm nhị phân:

```
while (left <= right) {
    mid = (left + right) / 2;
    if (arr[mid] == x) return mid;
    else if (arr[mid] < x) left = mid + 1;
    else right = mid - 1;
}
```

Kích thước đầu vào: n (độ dài mảng). Bậc tăng trưởng: $O(\log n)$ (logarit).

3. Sắp xếp nổi bọt (Bubble sort):

```
for (int i = 0; i < n-1; i++)
    for (int j = 0; j < n-i-1; j++)
        if (arr[j] > arr[j+1]) swap(arr[j], arr[j+1]);
```

Kích thước đầu vào: n (độ dài mảng). Bậc tăng trưởng: $O(n^2)$ (bậc hai).

4. Tháp Hà Nội (đệ quy):

```
void thapHaNoi(int n, char a, char b, char c) {
    if (n == 1) return;
    thapHaNoi(n-1, a, c, b);
    thapHaNoi(n-1, c, b, a);
}
```

Kích thước đầu vào: n (số đĩa). Bậc tăng trưởng: $O(2^n)$ (hàm mũ).

Bài tập 2: Mối quan hệ giữa các độ phức tạp dùng ký hiệu tiệm cận

Các ký hiệu tiệm cận (O , Ω , Θ , o , ω) dùng để mô tả mối quan hệ giữa các hàm biểu diễn độ phức tạp thuật toán.

Định nghĩa

- $f(n) = O(g(n))$: Tồn tại $c > 0, n_0 > 0$ sao cho $0 \leq f(n) \leq c \cdot g(n)$ với mọi $n \geq n_0$. (Chặn trên)
- $f(n) = \Omega(g(n))$: Tồn tại $c > 0, n_0 > 0$ sao cho $0 \leq c \cdot g(n) \leq f(n)$ với mọi $n \geq n_0$. (Chặn dưới)
- $f(n) = \Theta(g(n))$: $f(n) = O(g(n))$ và $f(n) = \Omega(g(n))$. (Chặn chặt)
- $f(n) = o(g(n))$: $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$. (Chặn trên ngặt)
- $f(n) = \omega(g(n))$: $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$. (Chặn dưới ngặt)

Mối quan hệ giữa các độ phức tạp thường gặp

Với $n \rightarrow \infty$, ta có:

$$1 = o(\log n) = o(\sqrt{n}) = o(n) = o(n \log n) = o(n^2) = o(2^n) \\ \log n = o(\sqrt{n}), \quad \sqrt{n} = o(n), \quad n = o(n \log n), \quad n \log n = o(n^2), \quad n^2 = o(2^n)$$

Ví dụ về mối quan hệ

1. $5n + 3 = \Theta(n)$
2. $n^2 + 100n = O(n^3)$ và đồng thời $n^2 + 100n = \Omega(n^2)$
3. $n \log n = O(n^2)$ nhưng $n \log n \neq \Theta(n^2)$
4. $2^{n+1} = \Theta(2^n)$
5. $\log(n!) = \Theta(n \log n)$ (theo xấp xỉ Stirling)
6. $n^{0.001}$ và $\log n$: thực tế $\log n$ tăng chậm hơn $n^{0.001}$ nên:

$$\lim_{n \rightarrow \infty} \frac{\log n}{n^{0.001}} = 0 \quad \Rightarrow \quad \log n = o(n^{0.001})$$

Suy ra $n^{0.001} = \omega(\log n)$.

Bảng tóm tắt mối quan hệ (đúng với n lớn)

$f(n)$	$g(n)$	$f = O(g)$	$f = \Omega(g)$	$f = \Theta(g)$	$f = o(g)$
1000	1	có	không	không	không
$\log n$	$n^{0.1}$	có	không	không	có
n	$n \log n$	có	không	không	có
$n \log n$	n^2	có	không	không	có
n^2	$n \log n$	không	có	không	không
2^n	n^{100}	không	có	không	không

Câu 11: Nhẹ hơn hay nặng hơn?

Bài toán: Có $n > 2$ đồng xu, 1 đồng xu giả (không biết nhẹ hơn hay nặng hơn), biết đồng xu nào là giả. Cần xác định đồng xu giả nhẹ hơn hay nặng hơn với số lần cân $\Theta(1)$.

Thuật toán:

- Lấy đồng xu giả G và 2 đồng xu thật T_1, T_2 (lấy từ $n - 1$ đồng còn lại, vì $n > 2$ nên có ít nhất 2 đồng thật).
- Cân G với T_1 :
 - Nếu $G = T_1$: G là thật $\Rightarrow$ vô lý (đã biết G là giả) $\Rightarrow$ không xảy ra.
 - Nếu $G < T_1$: G nhẹ hơn.
 - Nếu $G > T_1$: G nặng hơn.

Chỉ cần **1 lần cân** $\Rightarrow \Theta(1)$.

Algorithm 1 Xác định đồng xu giả nhẹ hay nặng

$n > 2$ đồng xu, biết đồng xu giả G , các đồng còn lại là thật Kết luận "nhẹ hơn" hoặc "nặng hơn" Lấy hai đồng thật T_1, T_2 từ các đồng còn lại Cân G với T_1 $G < T_1$ "Đồng xu giả nhẹ hơn" $G > T_1$ "Đồng xu giả nặng hơn"

Câu 12: Cánh cửa trên tường

Bài toán: Đứng trước tường vô tận, cửa ở khoảng cách n bước (chưa biết), không biết hướng. Cần tìm cửa với số bước đi $O(n)$.

Thuật toán: Tìm kiếm theo cấp số nhân (Exponential search / "Alternating doubling")

1. Bắt đầu tại vị trí 0, đặt $k = 1$.
2. Lặp:
 - Đi sang phải k bước, kiểm tra cửa.
 - Nếu chưa thấy, quay về vị trí 0.
 - Đi sang trái k bước, kiểm tra cửa.
 - Nếu chưa thấy, quay về vị trí 0.
 - Nhân đôi k : $k = 2k$.

3. Dừng khi tìm thấy cửa.

Chứng minh độ phức tạp: Giả sử cửa cách vị trí ban đầu n bước (theo một hướng nào đó). Khi k đạt đến $2^{\lceil \log_2 n \rceil} \leq 2n$, thuật toán sẽ thăm hướng đó. Tổng số bước đi:

$$\sum_{i=0}^{\lceil \log_2 n \rceil} 2 \cdot 2^i \cdot 2 = 4 \cdot (2^{\lceil \log_2 n \rceil + 1} - 1) \leq 4(4n - 1) = O(n)$$

Tổng số bước $\leq 16n$ bước (cận trên thực tế)

Algorithm 2 Tìm cửa trên tường vô tận

Vị trí ban đầu 0, không biết hướng và khoảng cách n Đến được cửa
 $k \leftarrow 1$ $i \leftarrow 1$ k đi phải 1 bước thấy cửa $i \leftarrow 1$ k đi trái 1 bước thấy cửa
 $i \leftarrow 1$ k đi trái 1 bước thấy cửa $i \leftarrow 1$ k đi phải 1 bước $k \leftarrow 2k$

Bài tập 3: Sắp xếp chọn (Selection Sort)

Mã giả

Algorithm 3 Selection Sort

Mảng $a[1..n]$ Mảng a được sắp xếp tăng dần $i \leftarrow 1$ $n-1$ $minIndex \leftarrow i$
 $j \leftarrow i + 1$ n $a[j] < a[minIndex]$ $minIndex \leftarrow j$ Hoán đổi $a[i]$ và
 $a[minIndex]$

Tại sao chỉ cần $n - 1$ bước?

Sau $n - 1$ bước, $n - 1$ phần tử đầu đã đúng vị trí. Phần tử cuối cùng tự động đúng vì chỉ còn một vị trí trống.

Độ phức tạp

- Best case: $O(n^2)$ (vẫn phải tìm min ở mỗi bước)
- Worst case: $O(n^2)$
- Trung bình: $O(n^2)$

So sánh: Mọi trường hợp đều $O(n^2)$ vì luôn duyệt toàn bộ phần còn lại để tìm min.

Chứng minh tính đúng đắn bằng bất biến lặp

Bất biến lặp: Sau mỗi lần lặp i , đoạn $a[1..i]$ chứa i phần tử nhỏ nhất của mảng ban đầu và được sắp xếp tăng dần.

- **Khởi tạo:** $i = 1$, $a[1..1]$ hiển nhiên đúng.
- **Duy trì:** Giả sử đúng sau $i - 1$. Ở bước i , tìm $\min$ trong $a[i..n]$ và đưa về $a[i]$. Khi đó $a[1..i]$ gồm i phần tử nhỏ nhất và sắp xếp.
- **Kết thúc:** Khi $i = n - 1$, $a[1..n - 1]$ đúng, phần tử cuối tự động đúng $\Rightarrow$ mảng sắp xếp hoàn chỉnh.

Cài đặt và kiểm thử (mã Python)

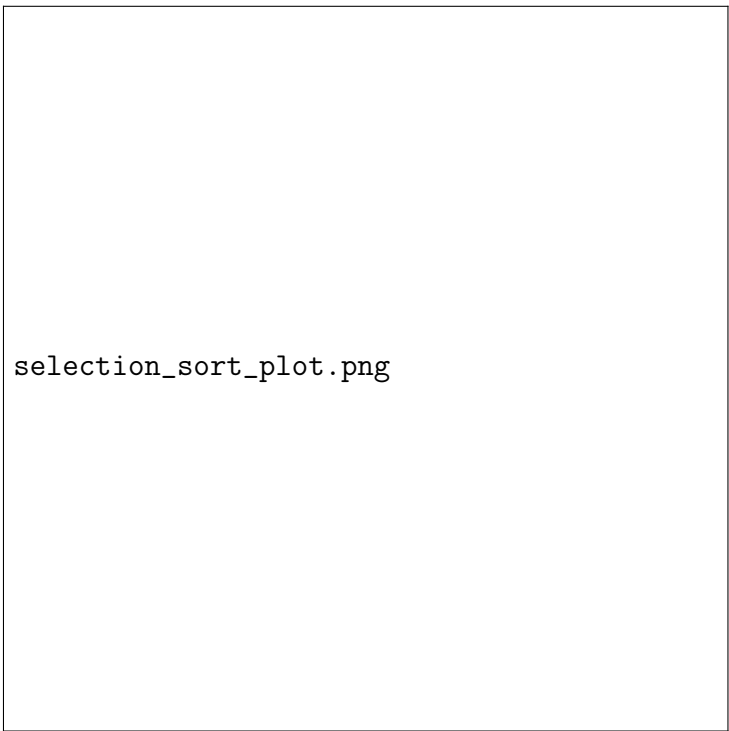
```
import random
import time
import matplotlib.pyplot as plt

def selection_sort(arr):
    n = len(arr)
    for i in range(n-1):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

# Đo thời gian
sizes = [100, 200, 500, 1000, 2000, 5000]
times = []
```

```
for size in sizes:
    arr = [random.randint(0, 10000) for _ in range(size)]
    start = time.time()
    selection_sort(arr.copy())
    end = time.time()
    times.append(end - start)

# Vẽ biểu đồ
plt.plot(sizes, times, 'o-')
plt.xlabel('Kích thước đầu vào (n)')
plt.ylabel('Thời gian (giây)')
plt.title('Selection Sort - Thời gian chạy')
plt.grid(True)
plt.show()
```



selection_sort_plot.png

Đề xuất hai thuật toán sắp xếp khác

1. Sắp xếp chèn (Insertion Sort)

Algorithm 4 Insertion Sort

$$\begin{aligned} i &\leftarrow 2 \quad n \quad key \leftarrow a[i] \quad j \leftarrow i - 1 \quad j \geq 1 \text{ AND } a[j] > key \quad a[j+1] \leftarrow a[j] \\ j &\leftarrow j - 1 \quad a[j+1] \leftarrow key \end{aligned}$$

Độ phức tạp: Best $O(n)$, Worst $O(n^2)$.

2. Sắp xếp nhanh (Quick Sort)

Algorithm 5 Quick Sort

$$\begin{aligned} &\text{QuickSort}(a, low, high) \quad low < high \quad pivotIndex \leftarrow \\ &\text{Partition}(a, low, high) \quad \text{QuickSort}(a, low, pivotIndex) - 1 \\ &\text{QuickSort}(a, pivotIndex + 1, high) \end{aligned}$$

Độ phức tạp: Best $O(n \log n)$, Worst $O(n^2)$, trung bình $O(n \log n)$.

So sánh ba thuật toán (đồ thị)

Sinh viên tự cài đặt và vẽ đồ thị so sánh thời gian chạy giữa Selection Sort, Insertion Sort và Quick Sort trên cùng bộ dữ liệu.